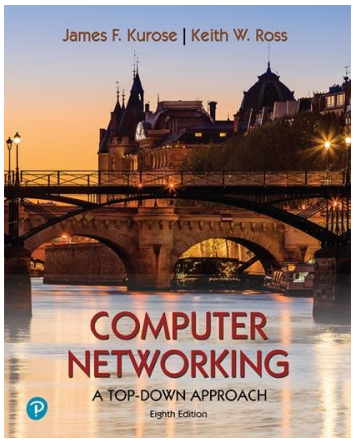
Wireshark Lab:
IP v8.0

Supplement to *Computer Networking: A Top-Down Approach, 8th ed.,* J.F. Kurose and K.W. Ross

"Tell me and I forget. Show me and I remember. Involve me and I understand." Chinese proverb

© 2005-2020, J.F Kurose and K.W. Ross, All Rights Reserved



In this lab, we'll investigate the IP protocol, focusing on the IP datagram. We'll do so by analyzing a trace of IP datagrams sent and received by an execution of the `traceroute` program (the `traceroute` program itself is explored in more detail in the Wireshark ICMP lab). We'll investigate the various fields in the IP datagram, and study IP fragmentation in detail.

Before beginning this lab, you'll probably want to review sections 1.4.3 in the text[1] and section 3.4 of RFC 2151 [ftp://ftp.rfc-editor.org/in-notes/rfc2151.txt] to update yourself on the operation of the `traceroute` program. You'll also want to read Section 4.3 in the text, and probably also have RFC 791 [ftp://ftp.rfc-editor.org/in-notes/rfc791.txt] on hand as well, for a discussion of the IP protocol.

1. Capturing packets from an execution of traceroute

In order to generate a trace of IP datagrams for this lab, we'll use the `traceroute` program to send datagrams of different sizes towards some destination, *X*. Recall that `traceroute` operates by first sending one or more datagrams with the time-to-live (TTL) field in the IP header set to 1; it then sends a series of one or more datagrams towards the same destination with a TTL value of 2; it then sends a series of datagrams towards the same destination with a TTL value of 3; and so on. Recall that a router must decrement the TTL in each received datagram by 1 (actually, RFC 791 says that the router must decrement the TTL by *at least* one). If the TTL reaches 0, the router returns an ICMP message (type 11 – TTL-exceeded) to the sending host. As a result of this behavior, a datagram with a TTL of 1 (sent by the host executing `traceroute`) will cause the router one hop away from the sender to send an ICMP TTL-exceeded message back to the sender; the datagram sent with a TTL of 2 will cause the router two hops

[1] References to figures and sections are for the 8th edition of our text, *Computer Networks, A Top-down Approach, 8th ed., J.F. Kurose and K.W. Ross, Addison-Wesley/Pearson, 2020.*

away to send an ICMP message back to the sender; the datagram sent with a TTL of 3 will cause the router three hops away to send an ICMP message back to the sender; and so on. In this manner, the host executing `traceroute` can learn the identities of the routers between itself and destination *X* by looking at the source IP addresses in the datagrams containing the ICMP TTL-exceeded messages.

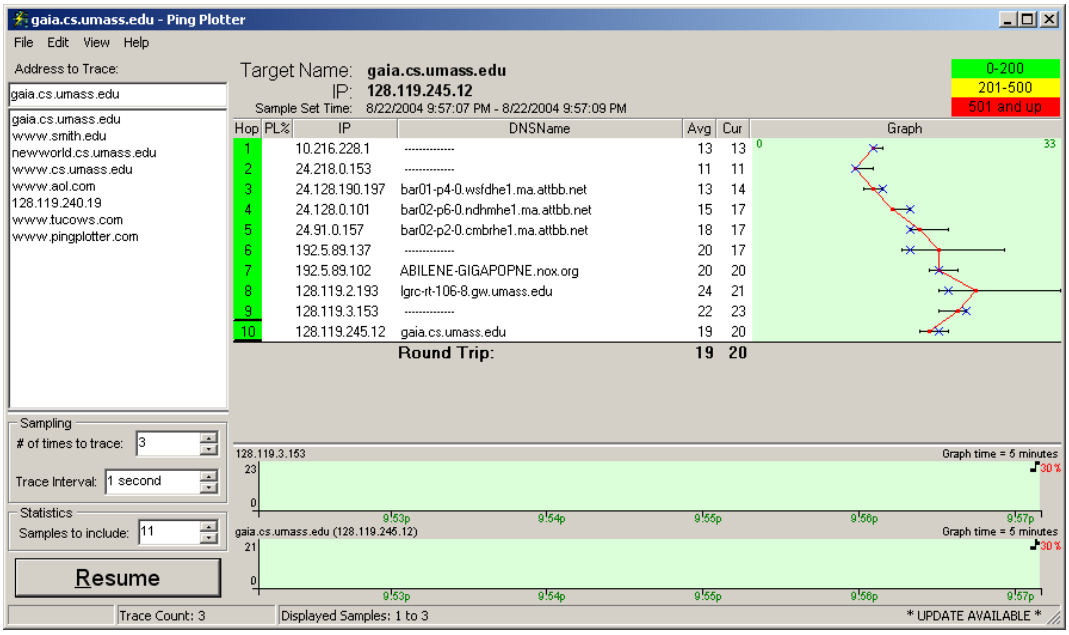
We'll want to run `traceroute` and have it send datagrams of various lengths.

- **Windows.** The `tracert` program (used for our ICMP Wireshark lab) provided with Windows does not allow one to change the size of the ICMP echo request (ping) message sent by the `tracert` program. A nicer Windows `traceroute` program is *pingplotter*, available both in free version and shareware versions at http://www.pingplotter.com. Download and install *pingplotter*, and test it out by performing a few traceroutes to your favorite sites. The size of the ICMP echo request message can be explicitly set in *pingplotter* by selecting the menu item *Edit-> Options->Packet Options* and then filling in the *Packet Size* field. The default packet size is 56 bytes. Once *pingplotter* has sent a series of packets with the increasing TTL values, it restarts the sending process again with a TTL of 1, after waiting *Trace Interval* amount of time. The value of *Trace Interval* and the number of intervals can be explicitly set in *pingplotter*.
- **Linux/Unix/MacOS.** With the Unix/MacOS `traceroute` command, the size of the UDP datagram sent towards the destination can be explicitly set by indicating the number of bytes in the datagram; this value is entered in the `traceroute` command line immediately after the name or address of the destination. For example, to send `traceroute` datagrams of 2000 bytes towards gaia.cs.umass.edu, the command would be:

```
%traceroute gaia.cs.umass.edu 2000
```

Do the following:

- Start up Wireshark and begin packet capture *(Capture->Start)* and then press *OK* on the Wireshark Packet Capture Options screen (we'll not need to select any options here).
- If you are using a Windows platform, start up *pingplotter* and enter the name of a target destination in the "Address to Trace Window." Enter 3 in the "# of times to Trace" field, so you don't gather too much data. Select the menu item *Edit->Advanced Options->Packet Options* and enter a value of 56 in the *Packet Size* field and then press OK. Then press the Trace button. You should see a *pingplotter* window that looks something like this:



Next, send a set of datagrams with a longer length, by selecting *Edit->Advanced Options->Packet Options* and enter a value of 2000 in the *Packet Size* field and then press OK. Then press the Resume button.

Finally, send a set of datagrams with a longer length, by selecting *Edit->Advanced Options->Packet Options* and enter a value of 3500 in the *Packet Size* field and then press OK. Then press the Resume button.

Stop Wireshark tracing.

- If you are using a Unix or Mac platform, enter three `traceroute` commands, one with a length of ~~56~~ 72 bytes, one with a length of 2000 bytes, and one with a length of 3500 bytes.
  traceroute -I gaia.cs.umass.edu 72
  (-I: Use ICMP ECHO for probes)

  Stop Wireshark tracing.

If you are unable to run Wireshark on a live network connection, you can download a packet trace file that was captured while following the steps above on one of the author's Windows computers[2]. You may well find it valuable to download this trace even if

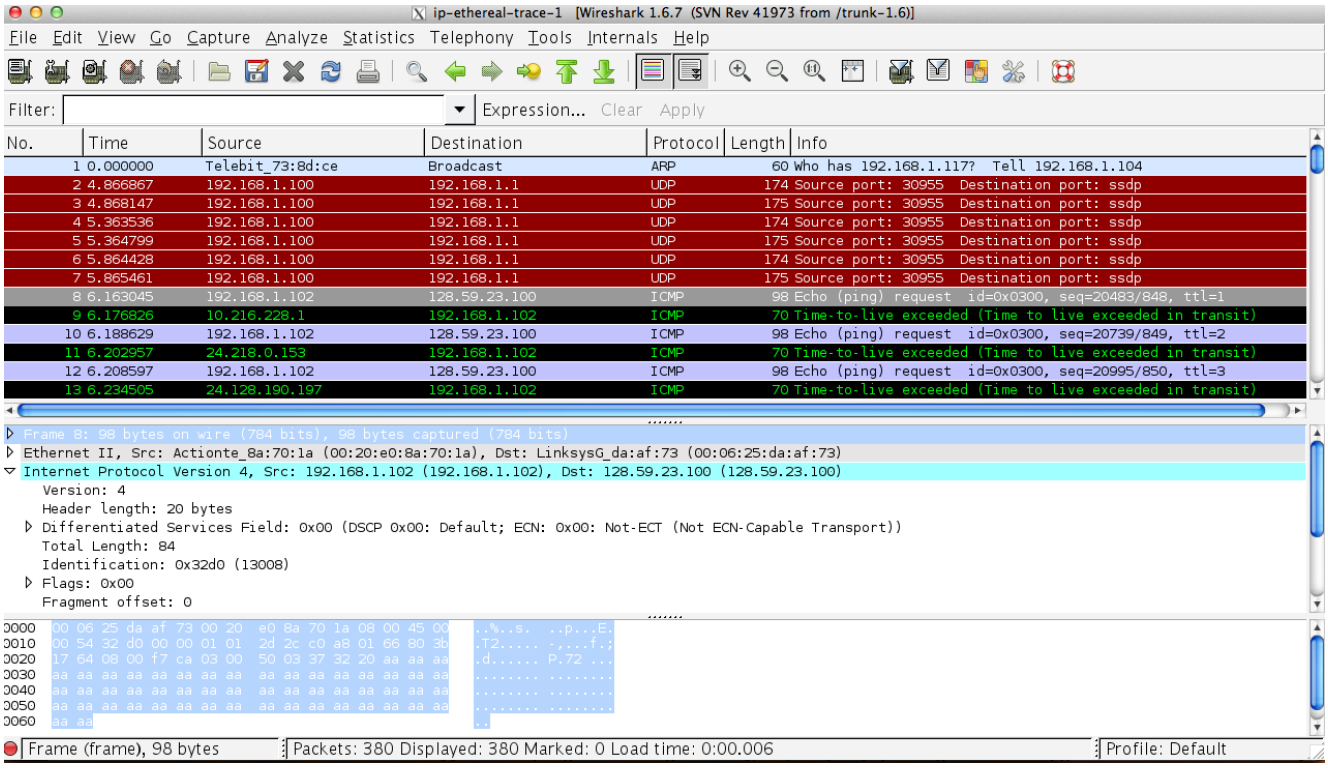
[2] Download the zip file http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces.zip and extract the file *ip-ethereal-trace-1*. The traces in this zip file were collected by Wireshark running on one of the author's computers, while performing the steps indicated in the Wireshark lab. Once you have downloaded the trace, you can load it into Wireshark and view the trace using the *File* pull down menu, choosing *Open*, and then selecting the ip-ethereal-trace-1 trace file.

you've captured your own trace and use it, as well as your own trace, when you explore the questions below.

2. A look at the captured trace

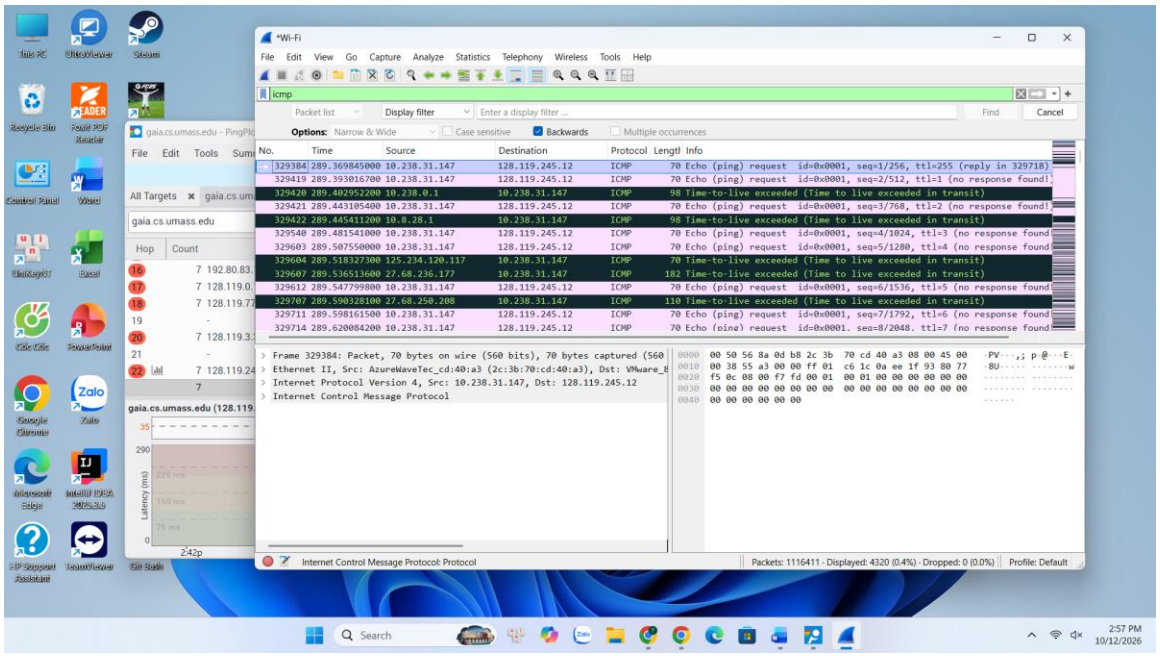
In your trace, you should be able to see the series of ICMP Echo Request (in the case of Windows machine) or the UDP segment (in the case of Unix) sent by your computer and the ICMP TTL-exceeded messages returned to your computer by the intermediate routers. In the questions below, we'll assume you are using a Windows machine; the corresponding questions for the case of a Unix machine should be clear. Whenever possible, when answering a question below you should hand in a printout of the packet(s) within the trace that you used to answer the question asked. When you hand in your assignment, annotate the output so that it's clear where in the output you're getting the information for your answer (e.g., for our classes, we ask that students markup paper copies with a pen, or annotate electronic copies with text in a colored font).To print a packet, use *File->Print*, choose *Selected packet only*, choose *Packet summary line,* and select the minimum amount of packet detail that you need to answer the question.

1. Select the first ICMP Echo Request message sent by your computer, and expand the Internet Protocol part of the packet in the packet details window.



What is the IP address of your computer?

My computer IP address: 10.238.31.147



2. Within the IP packet header, what is the value in the upper layer protocol field?
   The value in the upper layer protocol field is **ICMP (1)**
3. How many bytes are in the IP header? How many bytes are in the payload *of the IP datagram*? Explain how you determined the number of payload bytes.
   IP header: 20 bytes
   The payload bytes = Total length – IP header = 56 – 20 = 36 bytes
4. Has this IP datagram been fragmented? Explain how you determined whether or not the datagram has been fragmented.
   No, this IP datagram has not been fragmented.
   I determined this by examining the Flags and Fragment offset fields in the IP header. Specifically, the "More fragments" bit is set to 0 (Not set), and the "Fragment offset" is 0.

Next, sort the traced packets according to IP source address by clicking on the *Source* column header; a small downward pointing arrow should appear next to the word *Source*. If the arrow points up, click on the *Source* column header again. Select the first ICMP Echo Request message sent by your computer, and expand the Internet Protocol portion in the "details of selected packet header" window. In the "listing of captured packets" window, you should see all of the subsequent ICMP messages (perhaps with additional interspersed packets sent by other protocols running on your computer) below this first ICMP. Use the down arrow to move through the ICMP messages sent by your computer.

5. Which fields in the IP datagram *always* change from one datagram to the next within this series of ICMP messages sent by your computer?
   The fields in the IP datagram that always change from one datagram to the next within this series of ICMP messages are:

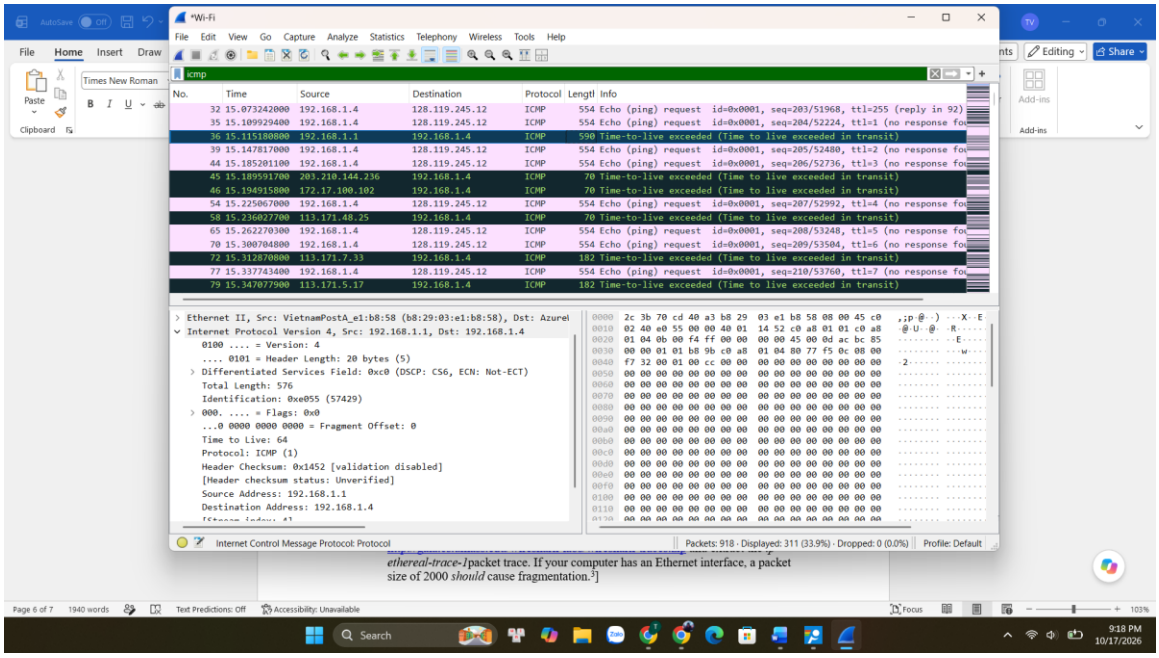
- **Identification:** This value strictly increments for every new IP datagram sent by the host to uniquely identify each packet.

- **Header Checksum:** Because the Identification field changes in every single packet, the header checksum must be mathematically recalculated every time to reflect the new header content.

6. Which fields stay constant? Which of the fields *must* stay constant? Which fields must change? Why?
   The fields that stay constant include Version, Header Length, Differentiated Services, Protocol, Source IP Address, and Destination IP Address. (Flags and Fragment Offset also remain constant as long as the packet size doesn't change). The Source IP Address, Destination IP Address, Version, and Protocol *must* stay constant.
   The Identification and Header Checksum fields *must* change from one datagram to the next.
7. Describe the pattern you see in the values in the Identification field of the IP datagram
   The pattern I see is that the values in the Identification field follow a strictly increasing sequence

Next (with the packets still sorted by source address) find the series of ICMP TTL-exceeded replies sent to your computer by the nearest (first hop) router.

8. What is the value in the Identification field and the TTL field?
   Identification: 0xe055 (57429)
   Time to Live: 64



9. Do these values remain unchanged for all of the ICMP TTL-exceeded replies sent to your computer by the nearest (first hop) router? Why?
   TTL remains unchanged because the router always initializes its newly generated packets with the same default TTL value (e.g., 64). Since the physical and logical distance from this first-hop router back to your computer is always exactly one hop, the packet always arrives with the exact same TTL value.

The **Identification** field must change because every ICMP TTL-exceeded reply is a brand new, distinct IP datagram generated by the router. The router's network stack must assign a unique, incrementing Identification number to every new datagram it creates to handle potential IP fragmentation properly.

Fragmentation

Sort the packet listing according to time again by click
ing on the *Time* column.

10. Find the first ICMP Echo Request message that was sent by your computer after you changed the *Packet Size* in *pingplotter* to be 2000. Has that message been fragmented across more than one IP datagram? [Note: if you find your packet has not been fragmented, you should download the zip file http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces.zip and extract the *ip-ethereal-trace-1*packet trace. If your computer has an Ethernet interface, a packet size of 2000 *should* cause fragmentation.[3]]
    Yes, the message has been fragmented across more than one IP datagram.
11. Print out the first fragment of the fragmented IP datagram. What information in the IP header indicates that the datagram been fragmented? What information in the IP header indicates whether this is the first fragment versus a latter fragment? How long is this IP datagram?
    The Flags field indicates this. Specifically, the "More fragments" bit is set to 1 (Set), which means there are additional fragments of this original datagram following it.
    The Fragment Offset field provides this information. For the first fragment, the offset is exactly 0, meaning this datagram carries the very beginning of the original data.
    The Total Length of this first IP datagram fragment is 1500 bytes (20 bytes of IP header + 1480 bytes of data payload).
12. Print out the second fragment of the fragmented IP datagram. What information in the IP header indicates that this is not the first datagram fragment? Are the more fragments? How can you tell?
    The Fragment Offset field indicates this. For this second fragment, the offset value is strictly greater than 0
    No, there are no more fragments.
    I can tell because the "More fragments" bit in the Flags field is set to 0 (Not set).
13. What fields change in the IP header between the first and second fragment?

---

[3] The packets in the *ip-ethereal-trace-1* trace file in http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces.zip are all less that 1500 bytes. This is because the computer on which the trace was gathered has an Ethernet card that limits the length of the maximum IP packet to 1500 bytes (40 bytes of TCP/IP header data and 1460 bytes of upper-layer protocol payload). This 1500 byte value is the standard maximum length allowed by Ethernet. If your trace indicates a datagram longer 1500 bytes, and your computer is using an Ethernet connection, then Wireshark is reporting the wrong IP datagram length; it will likely also show only one large IP datagram rather than multiple smaller datagrams.. This inconsistency in reported lengths is due to the interaction between the Ethernet driver and the Wireshark software. We recommend that if you have this inconsistency, that you perform this lab using the *ip-ethereal-trace-1* trace file.

Total Length**:** The first fragment is filled to the maximum transmission limit (e.g., 1500 bytes), while the second fragment only carries the remaining leftover data, resulting in a smaller total length.

Flags: This bit is set to 1 in the first fragment and 0 in the second (final) fragment.

Fragment Offset: The offset is 0 for the first fragment, and strictly greater than 0 for the second fragment.

Header Checksum: Because the Total Length, Flags, and Fragment Offset values are different, the checksum algorithm yields a different result for each fragment's header.

Now find the first ICMP Echo Request message that was sent by your computer after you changed the *Packet Size* in *pingplotter* to be 3500.

14. How many fragments were created from the original datagram?
    There are 2 fragments were created from the original datagram.
15. What fields change in the IP header among the fragments?
    Fragment Offset: Each fragment has a different offset value to indicate its specific position within the original unfragmented datagram.
    Flags (More fragments bit): This bit is set to 1 for all fragments except the very last one, where it is set to 0.
    Total Length: The intermediate fragments are usually filled to the maximum MTU size, while the final fragment usually has a smaller length containing just the leftover data.
    Header Checksum: Because the Offset, Flags, and Length fields vary among the fragments, the checksum must be recalculated and will be unique for each fragment's header.